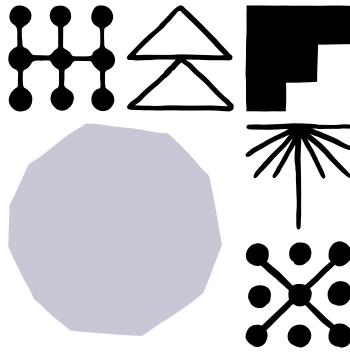


面向 AI Agent 的高效上下文工程

Effective context engineering for AI agents

Anthropic · 2025-09-29 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



上下文是 AI Agent 的关键资源，但也是有限资源。本文将探讨如何有效筛选、组织和管理支撑 Agent 运行的上下文。

在提示词工程成为应用 AI 领域关注焦点数年之后，一个新术语开始受到重视：**上下文工程**。利用语言模型构建应用，正越来越少地着眼于为提示词寻找恰当的字词和短语，而更多地转向回答一个更广泛的问题：“怎样组织上下文，最有可能让模型产生我们期望的行为？”

上下文是指从大语言模型（LLM）采样生成内容时所包含的一组 token。这里的**工程**问题，是在 LLM 固有约束下优化这些 token 的效用，从而稳定地获得期望结果。要有效驾驭 LLM，往往需要从**上下文的角度**思考，也就是说，要考虑 LLM 在任一时刻能够获得的完整状态，以及这种状态可能引发哪些行为。

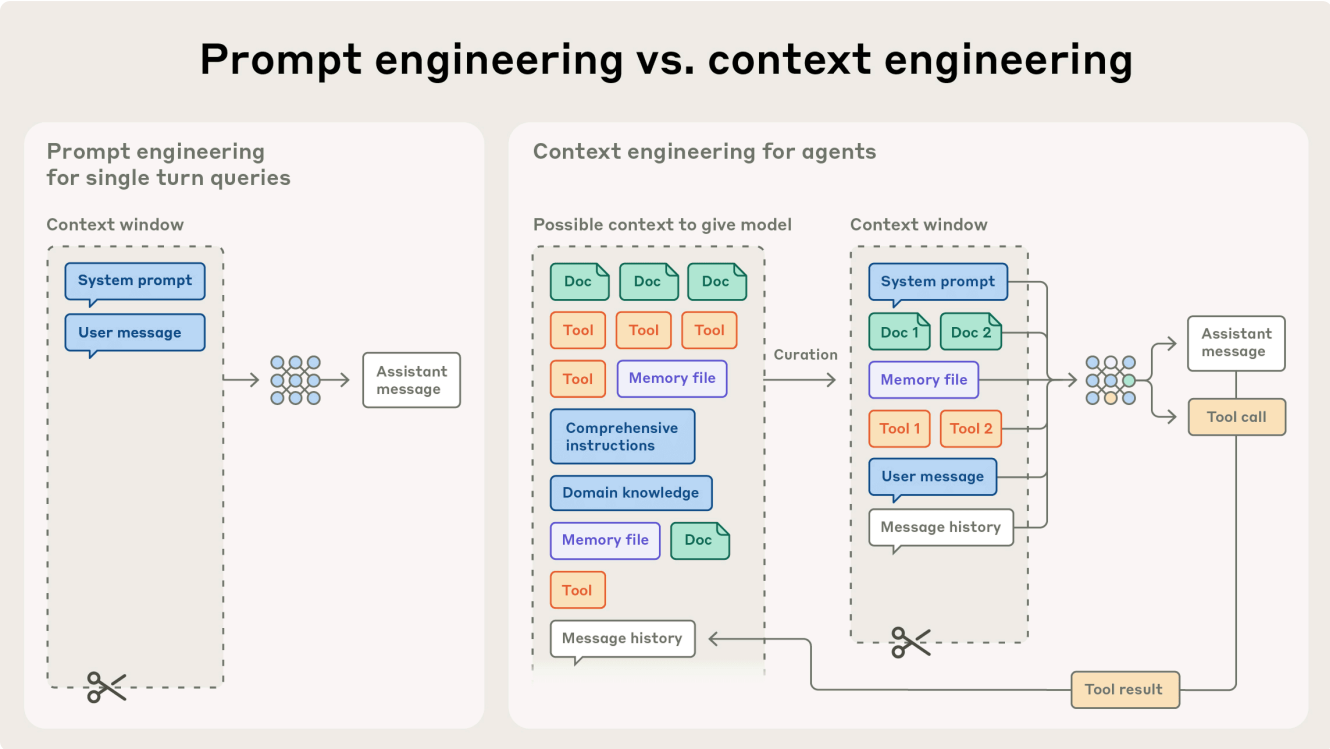
本文将探索上下文工程这门新兴技艺，并提供一个更完善的思维模型，帮助构建行为可引导、工作有效的 Agent。

上下文工程与提示词工程

在 Anthropic，我们认为上下文工程是提示词工程的自然延伸。提示词工程，是指为取得最佳结果而编写和组织 LLM 指令的方法；相关概述及实用的提示词工程策略，可参见我们的文档。**上下文工程**则是指在 LLM 推理期间筛选、组织并维护最佳 token（信息）集合的一系列策略，也包括提示词之外可能进入上下文的所有其他信息。

在使用 LLM 进行工程开发的早期，提示词是 AI 工程工作中最大的一部分，因为除日常聊天交互以外，大多数使用场景都需要针对单次分类或文本生成任务优化提示词。顾名思义，提示词工程主要关注如何编写有效的提示词，尤其是系统提示词。但随着我们开始构建能力更强、需要经过多轮推理并持续更长时间运行的 Agent，就需要管理整个上下文状态的策略，包括系统指令、工具、模型上下文协议（MCP）、外部数据、消息历史等。

循环运行的 Agent 会产生越来越多的、可能与下一轮推理有关的数据，因此必须反复筛选和提炼这些信息。上下文工程就是从不断变化的全部候选信息中，挑选出应该进入有限上下文窗口的内容的艺术与科学。



In contrast to the discrete task of writing a prompt, context engineering is iterative and the curation phase happens each time we decide what to pass to the model.

为什么上下文工程对于构建能力强大的 Agent 至关重要

尽管 LLM 速度很快，能够处理的数据量也越来越大，但我们观察到，它们和人类一样，在某个程度上也会失去专注或陷入混乱。“大海捞针”式基准测试的研究揭示了上下文腐化（context rot）这一现象：随着上下文窗口中的 token 数量增加，模型准确回忆其中信息的能力会下降。

尽管有些模型的性能下降比另一些更缓和，但所有模型都表现出这一特征。因此，必须将上下文视为边际收益递减的有限资源。正如人类的工作记忆容量有限，LLM 在解析大量上下文时，也要消耗自身的“注意力预算”。每加入一个新 token，都会消耗一部分预算，这使得我们更有必要仔细筛选和组织 LLM 可用的 token。

这种注意力的稀缺性源于 LLM 的架构约束。LLM 基于 Transformer 架构，使每个 token 都能够在整个上下文中关注其他每个 token。这意味着， n 个 token 会产生 n^2 个两两关系。

随着上下文长度增加，模型捕捉这些两两关系的能力会被摊薄，从而在上下文规模和注意力集中程度之间形成天然的矛盾。此外，模型的注意力模式是在训练数据分布中形成的，而训练数据中的短序列通常比长序列更常见。这意味着，模型处理跨越整个上下文的依赖关系时，经验更少，专门用于此类关系的参数也更少。

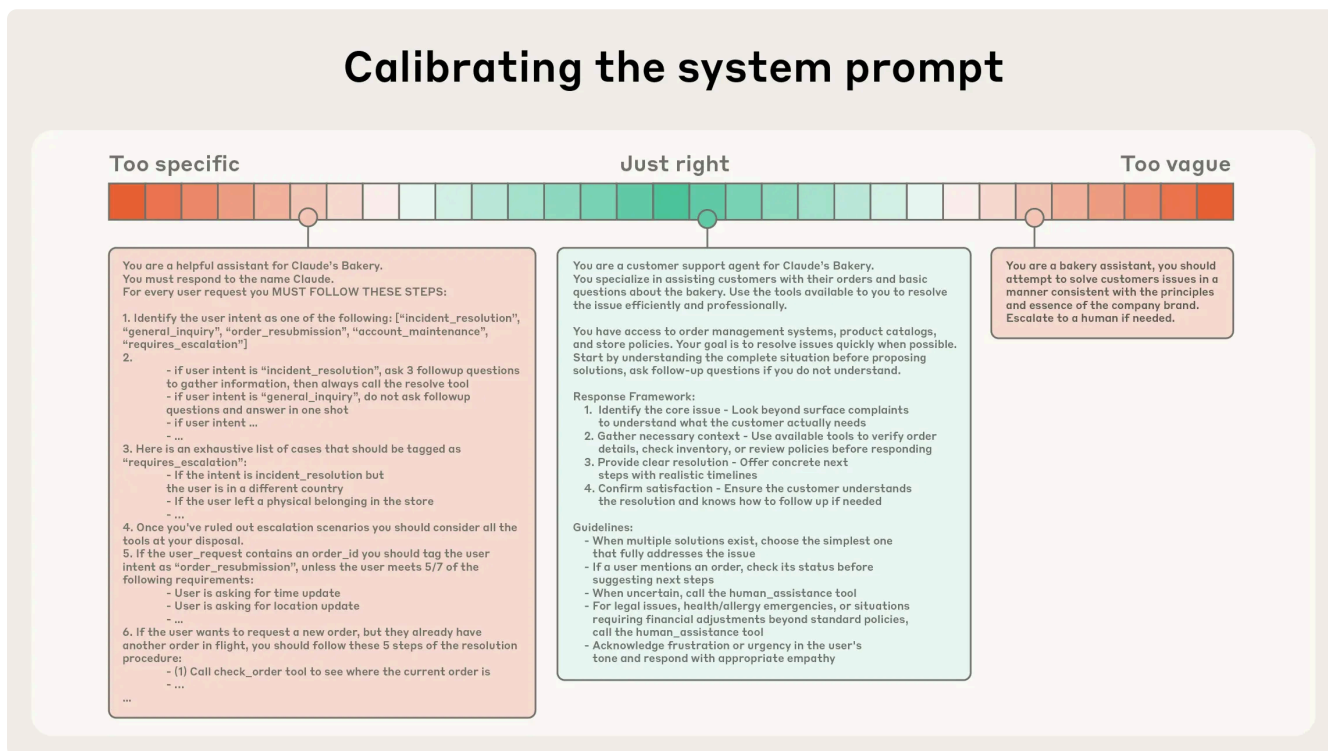
位置编码插值等技术，通过将更长的序列适配到模型最初训练时较小的上下文范围，使模型能够处理更长序列，但也会在一定程度上削弱模型对 token 位置的理解。这些因素带来的是逐渐下降的性能曲线，而不是陡然跌落的断崖：模型在较长上下文中仍然能力很强，但与较短上下文相比，信息检索和长距离推理的精度可能有所下降。

这些现实条件意味着，要构建能力强大的 Agent，经过审慎设计的上下文工程必不可少。

有效上下文的组成

鉴于 LLM 受有限注意力预算的约束，良好的上下文工程意味着：找到一组尽可能小的高信息量 token，使得某个期望结果的概率最大化。践行这一点，说起来远比做起来容易。下面我们将按上下文的不同组成部分，说明这条指导原则在实践中意味着什么。

系统提示词应当极其清晰，使用简洁、直接的语言，在适合 Agent 的抽象层次上表达想法。合适的抽象层次，位于两种常见失败模式之间恰到好处的区域。一种极端是，工程师在提示词中硬编码复杂、脆弱的逻辑，以便让 Agent 精确地按照指定方式行动。这种做法会导致系统脆弱，并随着时间推移增加维护复杂度。另一种极端是，工程师有时只提供模糊的高层指导，既未向 LLM 提供有关期望输出的具体信号，又或者错误地假设双方已经共享了背景信息。最佳抽象层次应取得平衡：既足够具体，能够有效引导行为；又足够灵活，能够为模型提供有力的启发式原则来指导行动。



At one end of the spectrum, we see brittle if-else hardcoded prompts, and at the other end we see prompts that are overly general or falsely assume shared context.

我们建议将提示词组织为不同部分，例如 `<background_information>`、`<instructions>`、`## Tool guidance`、`## Output description` 等，并使用 XML 标签或 Markdown 标题等方式划分这些部分。不过，随着模型能力增强，提示词的具体格式可能正变得不那么重要。

无论决定如何组织系统提示词，都应力求用最少的一组信息，完整说明你期望的行为。请注意，最少并不意味着简短；你仍然需要在一开始就给 Agent 足够的信息，确保它遵循期望的行为。最好先使用目前可用的最佳模型测试一个最精简的提示词，观察它在任务上的表现，再根据初步测试中发现的失败模式，添加清晰的指令和示例来改进表现。

工具让 Agent 能够与环境交互，并在工作过程中引入新的额外上下文。工具定义了 Agent 与其信息空间和行动空间之间的契约，因此，工具促进效率提升至关重要：既要节省 token 的方式返回信息，也要鼓励 Agent 采取高效的行为。

在《借助 AI Agent，为 AI Agent 编写工具》中，我们讨论了如何构建让 LLM 容易理解、功能重叠尽可能少的工具。类似于设计良好的代码库中的函数，工具应当功能自包含，能够稳健地应对错误，并且将预期用途表达得极其清楚。输入参数同样应具有描述性、没有歧义，并能发挥模型的固有优势。

我们最常见到的失败模式之一，是工具集过于臃肿，覆盖了过多功能，或让“该用哪个工具”的决策变得模糊。如果人类工程师都无法明确说出某种情况下该使用哪个工具，就不能指望 AI Agent 做得更好。正如后文将讨论的，为 Agent 精选一组最小可用工具，也有助于在长时间交互中更可靠地维护和裁剪上下文。

提供示例，也称为少样本提示（few-shot prompting），是一种广为人知的最佳实践，我们仍然强烈建议采用。不过，团队常常会往提示词里塞入一长串边缘情况，试图说清 LLM 在某项任务中应该遵循的每一条可能的规则。我们不建议这样做。相反，我们建议精心挑选一组多样且具有代表性的示例，有效展现 Agent 应有的行为。对 LLM 来说，示例就是那些“胜过千言万语”的图画。

对于上下文的各个组成部分，包括系统提示词、工具、示例、消息历史等，我们总体的建议是：审慎设计，让上下文信息充分而又紧凑。下面深入讨论如何在运行时动态检索上下文。

上下文检索与 Agent 自主搜索

在《构建有效的 AI Agent》中，我们强调了基于 LLM 的工作流与 Agent 之间的区别。自那篇文章发表以来，我们逐渐倾向于采用一个简单的定义：Agent 就是在循环中自主使用工具的 LLM。

在与客户合作的过程中，我们看到整个领域逐渐趋向这一简单范式。随着底层模型能力提升，Agent 的自主程度也可以提高：更聪明的模型让 Agent 能够独立应对细节复杂的问题空间，并从错误中恢复。

如今，我们看到工程师设计 Agent 上下文的思路正在发生变化。目前，许多 AI 原生应用会在推理前采用某种基于嵌入向量的检索方式，找出重要上下文供 Agent 推理。随着领域转向更强调 Agent 自主行动的方法，越来越多团队开始用“即时按需”的上下文策略来增强这些检索系统。

采用“即时按需”方式构建的 Agent，不会提前预处理所有相关数据，而是维护轻量级标识符，例如文件路径、保存的查询、网页链接等，并在运行时通过工具使用这些引用，动态地把数据加载到上下文中。Anthropic 的 Agent 式编程解决方案 Claude Code 就采用这种方式，在大型数据库上执行复杂的数据分析。模型可以编写有针对性的查询，保存结果，并利用 head、tail 等 Bash 命令分析大量数据，而无需将完整的数据对象加载到上下文中。这种方式与人类认知相似：我们通常不会记住整个信息库，而是借助文件系统、收件箱和书签等外部组织与索引系统，按需检索相关信息。

除了提高存储效率，这些引用的元数据还提供了一种高效调整行为的机制，无论元数据是明确给出的，还是可以直观推断出的。对在文件系统中工作的 Agent 而言，`tests` 文件夹中的 `test_utils.py`，与 `src/core_logic/` 中的同名文件，意味着不同的用途。文件夹层级、命名约定和时间戳都提供了重要信号，帮助人类和 Agent 理解应当如何以及何时利用信息。

让 Agent 自主浏览和检索数据，也能实现渐进式披露，也就是说，让 Agent 通过探索逐步发现相关上下文。每次交互都会产生上下文，为下一次决策提供依据：文件大小提示复杂度；命名约定暗示用途；时间戳可以作为相关性的间接指标。Agent 可以逐层建立理解，只在工作记忆中保留必要信息，并通过记笔记的策略获得额外的持久存储。这种自主管理的上下文窗口，使 Agent 能够专注于相关的信息子集，而不会淹没在全面却可能无关的信息中。

当然，这里存在取舍：运行时探索比检索预先计算好的数据更慢。不仅如此，还需要有明确设计取舍且经过深思熟虑的 engineered 工作，确保 LLM 具备合适的工具和启发式方法，从而有效地浏览其信息空间。如果缺乏适当指导，Agent 可能会因误用工具、沿无效方向不断探索，或未能识别关键信息而浪费上下文。

在某些场景中，最有效的 Agent 可能采用混合策略：预先检索部分数据以提高速度，再自行决定是否进一步自主探索。合适的自主程度应如何划界，取决于具体任务。Claude Code 就采用了这种混合模式：

CLAUDE.md 文件会在一开始直接放入上下文，而 glob、grep 等基础操作则让它能够浏览环境，并即时按需检索文件，从而有效避开索引过时和复杂语法树带来的问题。

混合策略可能更适合内容变化较少的场景，例如法律或金融工作。随着模型能力提升，Agent 设计会越来越倾向于让智能模型自主发挥智能，逐步减少人工筛选和组织。考虑到这一领域进展迅速，“采用能奏效的最简单做法”，很可能仍是我们给基于 Claude 构建 Agent 的团队的最佳建议。

面向长程任务的上下文工程

长程任务要求 Agent 在一系列行动中保持连贯性、上下文以及围绕目标展开的行为，而这些行动累计产生的 token 数量会超出 LLM 的上下文窗口。对于需要连续工作数十分钟到数小时的任务，例如大型代码库迁移或全面的研究项目，Agent 需要专门的技术来突破上下文窗口大小的限制。

等待更大的上下文窗口，似乎是一个显而易见的策略。但在可预见的未来，各种大小的上下文窗口很可能都会面临上下文污染和信息相关性的问题，至少在追求最强 Agent 表现时是如此。为了让 Agent 在更长时间跨度内有效工作，我们开发了一些直接应对这些上下文污染限制的技术：压缩、结构化笔记，以及多 Agent 架构。

压缩

压缩是指对一段接近上下文窗口上限的对话进行总结，并以该摘要重新启动一个新的上下文窗口。压缩通常是上下文工程中用来提升长期连贯性的首要手段。其核心是高保真地提炼上下文窗口的内容，使 Agent 能够继续工作，同时尽可能少地损失表现。

例如，在 Claude Code 中，我们将消息历史交给模型，让它总结并压缩最关键的细节。模型保留架构决策、尚未解决的 bug 和实现细节，同时丢弃冗余的工具输出或消息。随后，Agent 可以凭借压缩后的上下文，以及最近访问的五个文件继续工作。这样，用户能够获得连续的使用体验，而不必担心上下文窗口的限制。

压缩的技巧在于选择保留什么、丢弃什么，因为过于激进的压缩可能丢失细微却关键的上下文，而这些信息的重要性往往要到后来才会显现。对于实现压缩系统的工程师，我们建议利用复杂的 Agent 执行轨迹，仔细调优提示词。首先尽可能提高召回率，确保压缩提示词能从执行轨迹中捕捉每一项相关信息；随后通过去除多余内容，迭代提高精确率。

一种容易着手清理的冗余内容，就是工具调用及其结果：如果一次工具调用已经位于消息历史的很早位置，Agent 为什么还需要再次看到其原始结果？工具结果清理是最安全、干预最轻的一类压缩方式之一，最近已作为功能在 Claude 开发者平台上推出。

结构化笔记

结构化笔记，也称为 Agent 记忆，是一种让 Agent 定期撰写笔记，并将笔记持久保存到上下文窗口之外的记忆中的技术。之后，需要时再将这些笔记取回到上下文窗口中。

这种策略以极小的开销提供持久记忆。无论是 Claude Code 创建待办事项列表，还是你的自定义 Agent 维护 NOTES.md 文件，这种简单模式都能让 Agent 跟踪复杂任务的进展，保留那些原本会在数十次工具调用后丢失的关键上下文和依赖关系。

Claude 玩《宝可梦》展示了记忆如何改变 Agent 在非编程领域的能力。Agent 在数千个游戏步骤中保持精确记录，跟踪目标，例如：“过去 1,234 步，我一直在 1 号道路训练宝可梦；目标是让皮卡丘提升 10 级，目前已提升 8 级。”即使没有任何关于记忆结构的提示，它也会绘制已探索区域的地图，记住已经解锁的关键成就，并记录战斗策略笔记，帮助自己学会面对不同对手时哪些攻击最有效。

上下文重置后，Agent 会读取自己的笔记，继续数小时的训练过程或地下城探索。这种跨越多次总结仍能保持的连贯性，使长程策略成为可能；如果仅将所有信息保留在 LLM 的上下文窗口内，就无法做到这一点。

作为 Sonnet 4.5 发布的一部分，我们在 Claude 开发者平台上推出了记忆工具的公开测试版，通过基于文件的系统，更方便地在上下文窗口之外存储和查阅信息。这让 Agent 能够随时间积累知识库，在不同会话之间保持项目状态，并参考之前的工作，而不必把一切都留在上下文中。

子 Agent 架构

子 Agent 架构提供了另一种绕开上下文限制的方法。与其让一个 Agent 尝试维护整个项目的状态，不如让专门的子 Agent 在干净的上下文窗口中处理聚焦的任务。主 Agent 依据高层计划进行协调，子 Agent 则承担深入的技术工作，或使用工具查找相关信息。每个子 Agent 都可能进行大量探索，使用数万个甚至更多 token，但只返回一份浓缩提炼后的工作摘要，通常为 1,000 至 2,000 个 token。

这种方法实现了清晰的职责分离：详细的搜索上下文隔离在子 Agent 内部，而主导 Agent 则专注于整合和分析结果。我们在《我们如何构建多 Agent 研究系统》中讨论过这一模式；在复杂研究任务上，它相较单 Agent 系统表现出显著提升。

如何在这些方法之间选择，取决于任务特点。例如：

- 对于需要大量来回交互的任务，压缩可以保持对话的连贯性；
- 对于具有明确里程碑的迭代开发，记笔记尤其有效；
- 对于并行探索能够带来收益的复杂研究和分析，多 Agent 架构更适合。

即使模型持续进步，在长时间交互中保持连贯性的挑战，仍将是构建更有效 Agent 的核心问题。

结语

上下文工程代表着我们利用 LLM 构建应用的方式发生了根本转变。随着模型能力增强，挑战不再只是精心打造一个完美提示词，还在于审慎选择每一步有哪些信息进入模型有限的注意力预算。无论是在长程任务中实现压缩、设计节省 token 的工具，还是让 Agent 即时按需探索环境，指导原则始终相同：找到最小的一组高信息量 token，使获得期望结果的概率最大化。

随着模型改进，本文概述的技术也将继续演进。我们已经看到，更聪明的模型更少需要通过工程设计细致规定其行为，这让 Agent 能够更加自主地运行。但即使能力不断增强，将上下文视为珍贵的有限资源，仍将是构建可靠、有效的 Agent 的核心。

现在即可在 Claude 开发者平台上开始实践上下文工程，并通过我们的记忆与上下文管理实践指南，获取实用技巧和最佳实践。

致谢

本文由 Anthropic 应用 AI 团队的 Prithvi Rajasekaran、Ethan Dixon、Carly Ryan 和 Jeremy Hadfield 撰写，团队成员 Rafi Ayub、Hannah Moran、Cal Rueb 和 Connor Jennings 参与贡献。特别感谢 Molly Vorwerck、Stuart Ritchie 和 Maggie Vo 的支持。